

Importing Messy Spreadsheets

Approaches for onboarding properties, owners, leases, maintenance, and providers from Excel and Google Sheets, and an assessment of Jev

Summary

Jev could help in one narrow slot, the classification and confidence-scoring steps, but it is not an ETL solution on its own. At your onboarding volumes it is probably not required. The sections below describe how to approach the whole import problem, then where Jev fits.

1. Core design: AI proposes, the operator approves

Your product brief already describes this: operator-mapped columns, review of validation and duplicates, create/link/skip confirmation, no silent merges, and no write-back to source sheets. The importer can be built as a pipeline around that contract.

1. **Read into a neutral grid.** Keep values, formats, merged cells, hidden rows and sheets, cell colors, comments, and cached formula results. Store the original file with sheet and row provenance for every imported record.
2. **Profile.** For each column, record sample values, fill rate, and how often values match patterns like phone, email, date, money, ZIP, and state.
3. **Find structure.** Detect tables, header rows, section titles, subtotals, blank rows, and notes.
4. **Propose a mapping.** Assign each sheet to an entity type and each column to a target field (or “keep as note” or “ignore”), with a confidence for each.
5. **Normalize values** (dates, money, phones, addresses, status words, multi-value cells). Use deterministic code where possible and AI for the fuzzy cases.
6. **Resolve relationships and duplicates** across owners, properties, units, leases, and providers, both within the file and against existing records.
7. **Review by exception.** Show a dry run, auto-accept high-confidence items, and queue the rest for the operator.
8. **Commit with a batch ID and undo.** Save the mapping as a reusable recipe for re-imports.

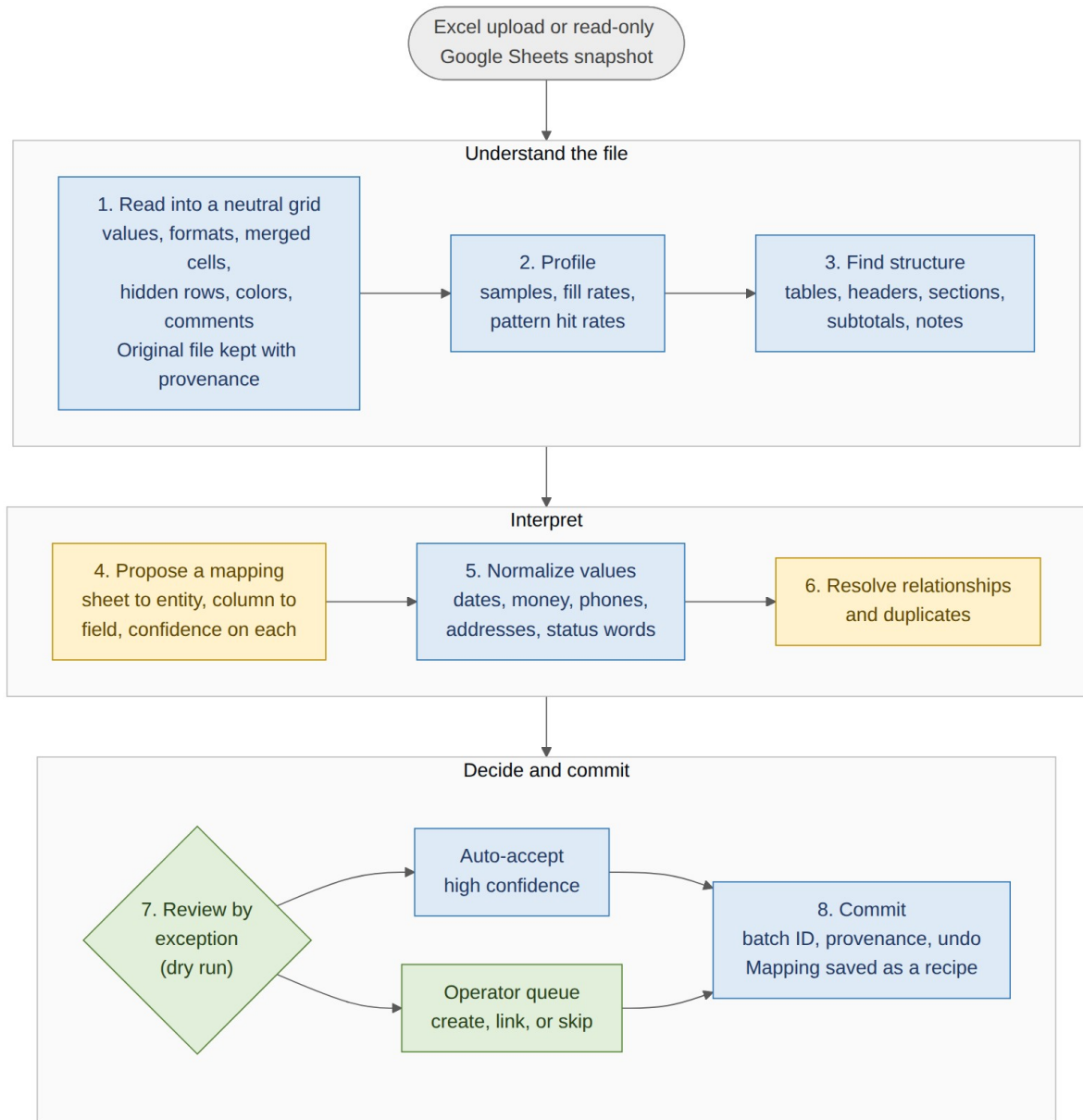


Figure 1. The import pipeline. Blue = automated code, amber = AI proposes with a confidence, green = operator decides, gray = source.

2. Tactics for unpredictable formats

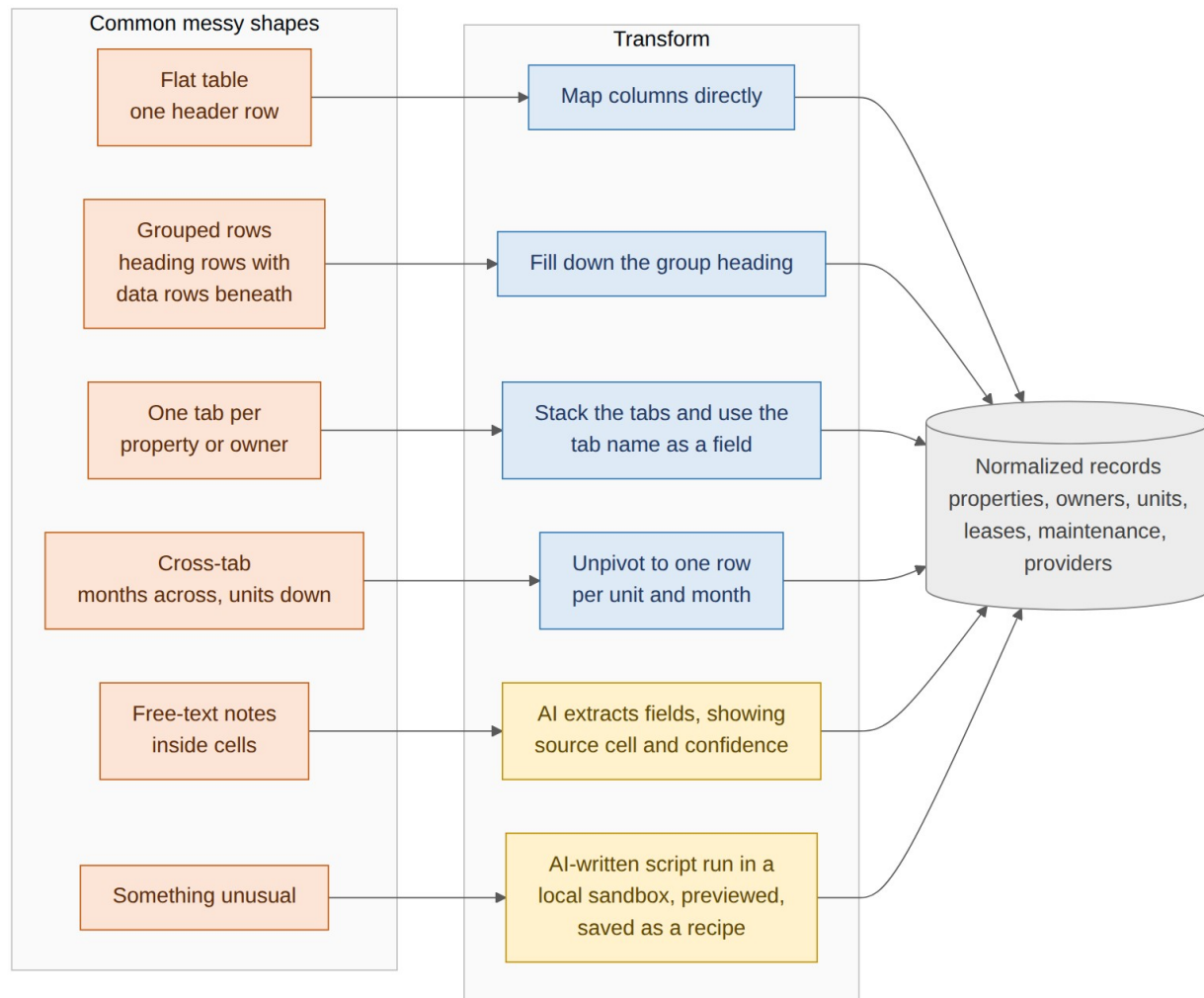


Figure 2. Common messy layouts and the transform each one needs. Orange = input shape, blue = rule-based transform, amber = AI-assisted transform.

- **Layout archetypes.** Most “random” sheets fall into a few shapes, each with its own transform: flat table, groups under an owner or property heading, one tab per property or owner, cross-tab (months across, units down), and free-text notes in cells.
- **AI-written transform scripts for odd layouts.** Run them locally in a sandbox with no network, show a preview, and save the script as the recipe.
- **Never drop data.** Unmapped columns become custom fields or notes.
- **Adapters for common sources.** Many customers export from Buildium, AppFolio, TenantCloud, or QuickBooks, which have more predictable formats. Verify the current export formats before building.
- **Stage the scope.** Do properties, owners, and providers first. Then units and current leases, then open maintenance and recent history. Skip balances and ledger history at first, which fits the brief’s “later scope.”

- **Concierge imports during pilots.** Log every quirk, charge a setup fee, and keep anonymized samples (with consent) as a regression test set.
- **Template fallback.** For the worst sheets, offer a simple template to paste into.

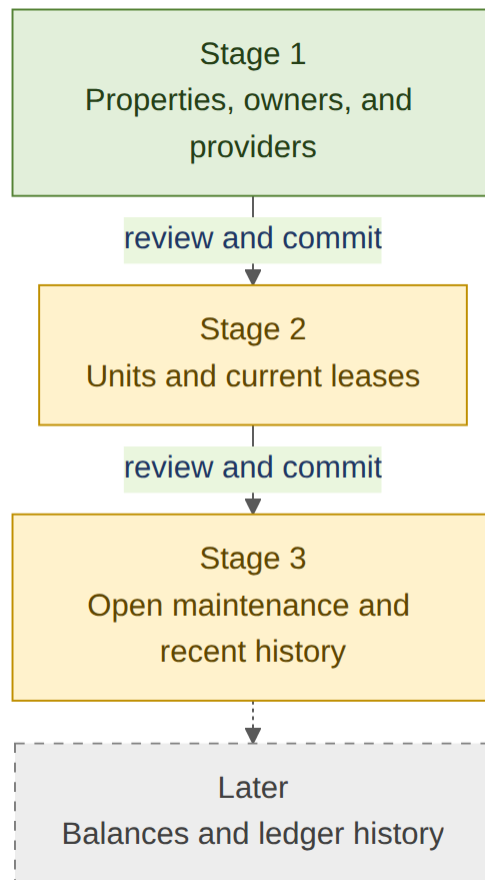


Figure 3. Staged scope. Green = matches the brief's initial intake, amber = later stages to add, gray dashed = skip at first.

3. Excel and Google Sheets specifics

- **Excel:** date serials (including the 1904 epoch), numbers stored as text (lost leading zeros on ZIPs and phones), merged cells, hidden sheets and rows, color-coded status (read the cell fills), legacy .xls, password-protected files, and CSV encodings.
- **Google Sheets:** keep it a read-only snapshot. The least privileged routes are having the user pick the file or export XLSX/CSV. If you use OAuth, choose the narrowest scope and check Google's current verification requirements.

4. Where Jev fits

From TypeSafe's launch blog post and documentation, Jev takes a state plus typed questions (Choice, Score, or yes/no "Noul") and returns typed answers with probabilities and confidence. It generates no text, and it is in early access.

Good fits	Poor fits
- Column to target field, given a header and sample values, with "unknown" as an option. - Row type: header, data, subtotal, blank, or note. - Sheet entity type. - Yes/no checks, such as "is this value a company or a person?" or "are these two records the same property?" - Calibrated confidence for deciding what to auto-accept and what to send to review.	- Writing transform code, unpivoting a cross-tab, or planning multi-step restructuring. - Splitting or rewriting free text, unless you generate candidate splits and it picks among them. - Explaining problems to the operator.

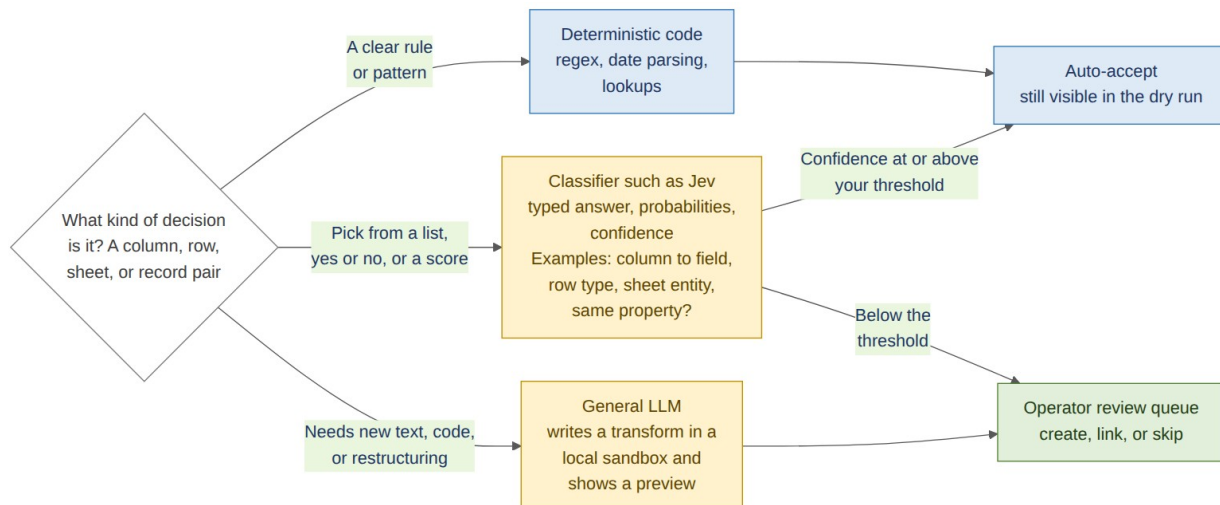


Figure 4. Who makes which decision. Blue = automated code, amber = AI, green = operator.

Is it worth it?

The headline advantages are speed and cost per call, and onboarding is a one-time job of hundreds to thousands of rows per customer, so those matter little. The useful parts would be guaranteed typed outputs and calibrated confidence.

Cautions

- The speed, cost, and accuracy numbers are the vendor's own evals, which they say are not independent.

- Sending tenant and owner data to a third-party cloud service cuts against your ownership contract. You would need consent, minimized samples or masking, and a data-flow screen entry.
- I did not see their retention or training terms, so check those.
- Early-stage vendor risk is real.

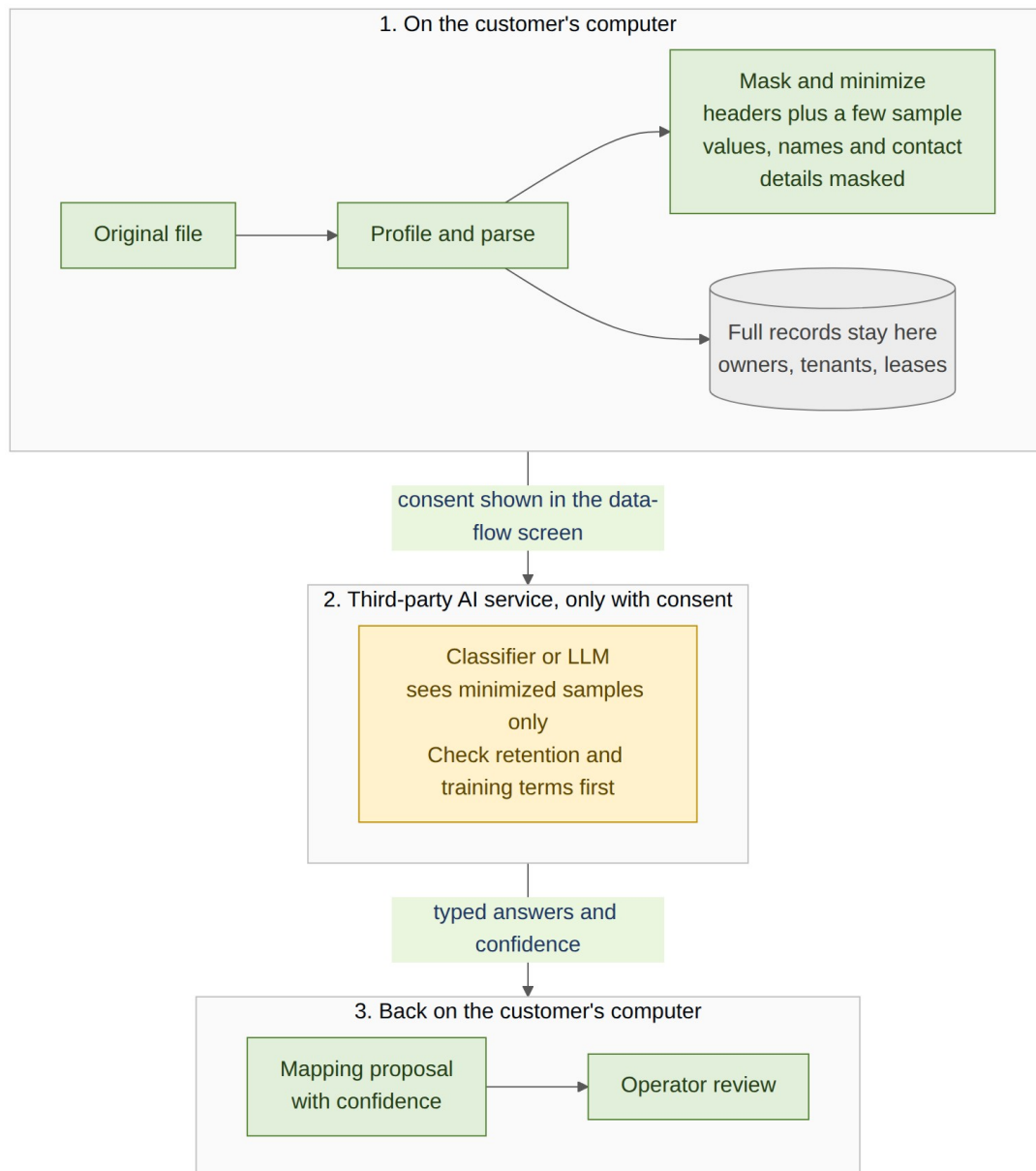


Figure 5. The data boundary when an outside AI service is used. Green = stays on the customer's computer, amber = third-party service.

Recommended next step

Put classification behind your own “decision provider” interface, then run a bake-off on 20–30 messy sheets comparing heuristics only, a general LLM with structured output, and Jev.

Compare correct mappings at a fixed review burden, whether stated confidence matches actual accuracy, cost, and privacy impact.

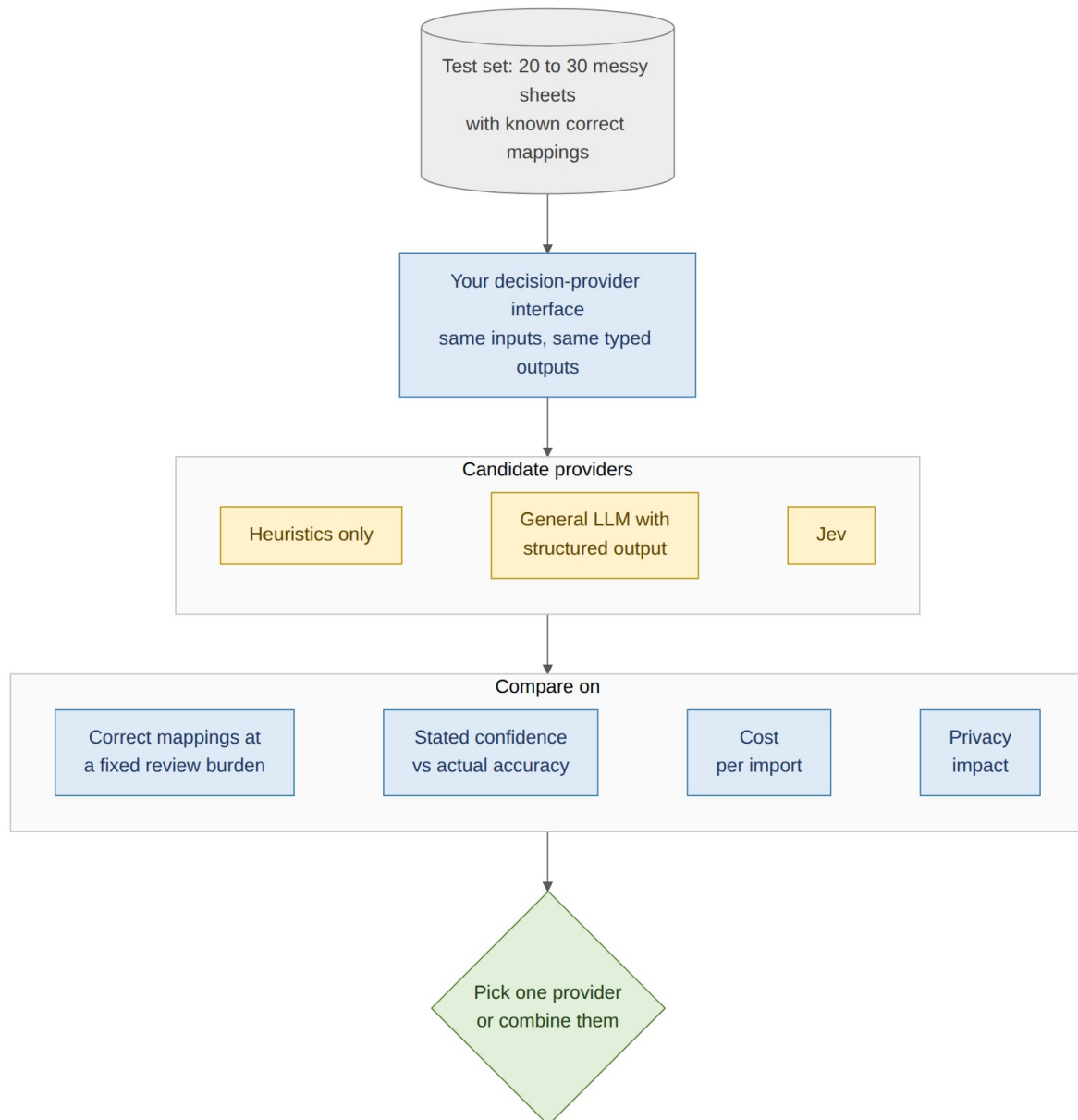


Figure 6. Bake-off for choosing a decision provider. Gray = test data, blue = your code and metrics, amber = candidate providers, green = decision.

5. How to measure success

Build a test set of messy sheets and track three numbers:

- The share of columns auto-mapped correctly at your confidence threshold (aim for 98% or better on auto-accepted items **[hypothesis]**).
- Operator corrections per import.
- Time from upload to commit.

Also track duplicate and error rates found after import.

Possible next deliverables

- A target schema and mapping spec for properties, owners, leases, maintenance, and providers.
- A test-set checklist for the bake-off.

Sources

- TypeSafe AI, “Introducing System One Models & Jev” (typesafe.ai/blog/introducing-system-one-models-and-jev).
- TypeSafe documentation, Introduction (docs.typesafe.ai/introduction).
- Project product brief (PRODUCT_BRIEF.md), section on initial intake for Excel and Google Sheets.